Secure File Exchange Platform

D4 — MIDTERM SYSTEM PRESENTATION & SECURITY REVIEW

- Arturo Alfaro Domínguez
- Karina Jezuleth Herrera Gallegos
- Dannae Sánchez Durán
- Javier de Jesús Infante Martínez

A Python-based prototype for securely exchanging confidential files through untrusted environments.



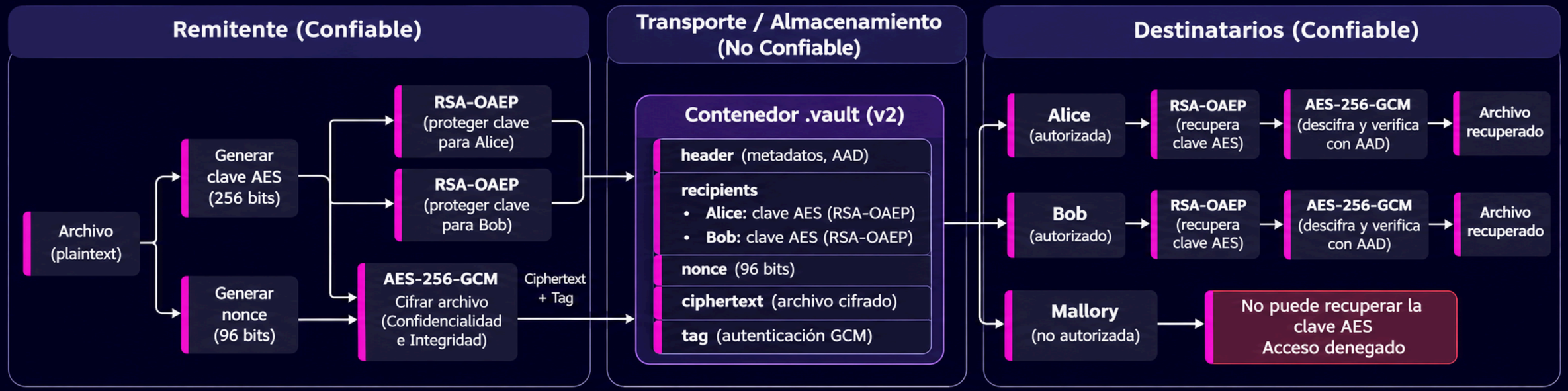
The Problem

- Shared files may be intercepted or modified.
- Unauthorized users may attempt to access confidential information.
- Multiple recipients need secure access to the same document.

Our Solution

- Authenticated file encryption using AES-256-GCM.
- Multi-recipient key protection using RSA-OAEP.
- Encrypted file packaging using the .vault container.
- Detection of unauthorized modifications.

System Architecture & Data Flow



Main Components

- Vault: File encryption and decryption operations.
- Key Management: Cryptographic key generation and recipient key handling.
- .vault Container (v2): Stores metadata, recipient-protected keys, nonce, ciphertext, and authentication tag.

Trust Boundaries

- Trusted: Sender and authorized recipient environments.
- Untrusted: Network, cloud storage, USB devices, and unverified containers.



Threat Model (D1)

PROTECTED ASSETS

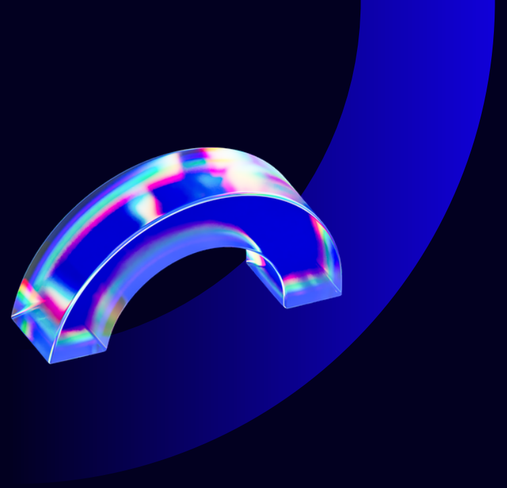
- Confidential documents
- Private cryptographic keys
- Public-key ownership associations
- Recipient metadata

ADVERSARY CAPABILITIES

- Intercept or copy encrypted containers.
- Modify ciphertext or metadata.
- Attempt unauthorized file recovery.

SECURITY ASSUMPTIONS

- Private keys remain protected.
- Recipient public keys are correctly associated with their owners.
- Trusted endpoints are not compromised.



Authenticated Encryption (D2)

AES–256–GCM

CRYPTOGRAPHIC PARAMETERS

- AES file key: 256 bits
- Nonce: 96 bits
- Encryption mode: Galois/Counter Mode (GCM)
- AAD: Authenticated metadata
- Authentication tag: Integrity verification

SECURITY PROPERTIES

- Confidentiality of file contents
- Detection of unauthorized modifications
- Authentication of protected data and metadata

Encryption Process

File + AES Key + Nonce + AAD → AES-256-GCM → Ciphertext + Tag

Hybrid Encryption & Sharing (D3)

AES-256-GCM + RSA-OAEP-SHA256

How It Works

1. Generate a random AES-256 file key.
2. Encrypt the file once using AES-GCM.
3. Protect the AES key separately with each recipient's RSA public key.
4. Store the ciphertext and protected keys in the .vault container.
5. Authorized recipients recover the AES key with their private keys.

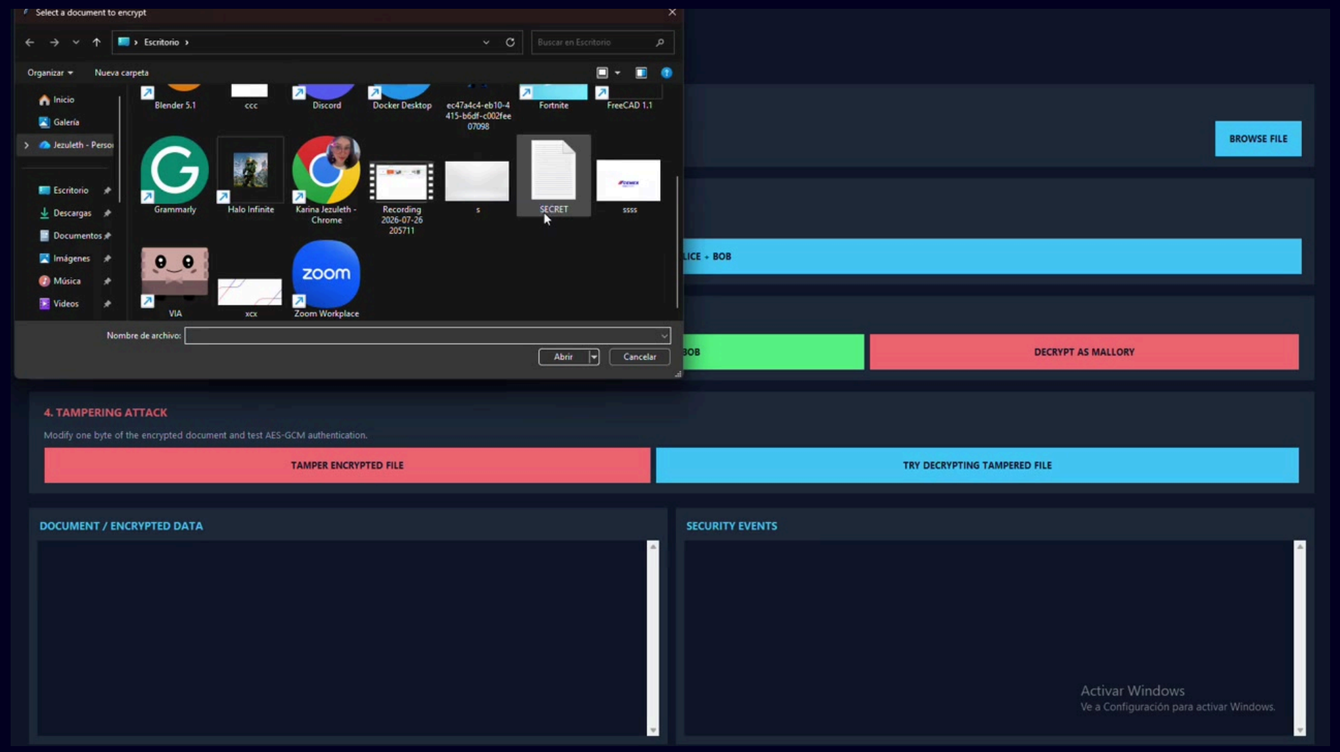
Cryptographic Details

- RSA key size: 3072 bits
- Key protection: RSA-OAEP with SHA-256
- Recipient identification: SHA-256 public-key fingerprints

Live Security Demonstration (D4)

Demonstration Scenarios

1. **Encryption:** Select a file and generate its encrypted .vault container.
2. **Alice:** Successfully decrypts the file.
3. **Bob:** Successfully decrypts the same container.
4. **Mallory:** Unauthorized access is denied.
5. **Tampering:** Modified encrypted content fails authentication (InvalidTag).



Security Review & Project Status

Implemented Features

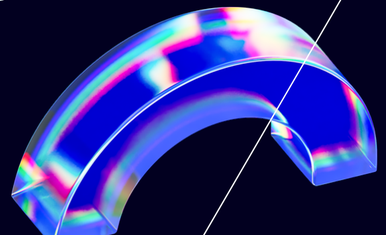
- D1: Architecture and Threat Model
- D2: AES-256-GCM Authenticated Encryption
- D3: Hybrid Multi-Recipient Encryption
- D4: Interactive Security Demonstration

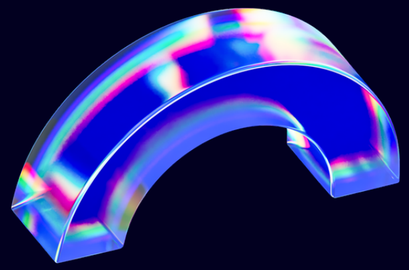
Testing

165 automated tests passed in the latest local validation.

Current Limitations

- Demo identities and keys are generated temporarily.
- Public-key authenticity remains a trust assumption.
- Sender identity is not independently verified through digital signatures.





Key Achievements

- Confidential files are protected through authenticated encryption.
- One encrypted file can be shared with multiple authorized recipients.
- Unauthorized recipients cannot recover the protected file key under the stated security assumptions.
- Unauthorized modifications to authenticated content are detected.

Conclusions

Technologies

Python · cryptography · AES-256-GCM · RSA-OAEP · SHA-256 · Tkinter · pytest

GitHub Repository

https://github.com/Jezuleth/Team4_Criptography2027-1

Team 4 — Cryptography 2027-1

Thank you!